

Ottimizzazione della Memoria nella Programmazione Dinamica per l'Allineamento di Sequenze

Iacopo Giansanti

September 2026

Indice

1	Introduzione	3
2	Definizione del Global Alignment problem e dei prerequisiti essenziali	3
2.1	Rappresentazione delle sequenze biologiche	3
2.2	Allineamento e modello di scoring	4
2.3	Programmazione dinamica	5
2.4	Longest Common Subsequence	6
2.5	Alignment graph	7
2.6	Global Alignment Problem	8
3	Algoritmo di Needleman-Wunsch	9
3.1	Ricorrenza, inizializzazione e traceback	9
3.2	Pseudocodice	10
3.3	Analisi della complessità	11
3.4	Implementazione	13
4	Algoritmo di Hirschberg	13
4.1	Ricorrenza, inizializzazione e traceback	13
4.2	Pseudocodice	13
4.3	Analisi della complessità	13
4.4	Implementazione	13
5	Metodologia sperimentale	13
5.1	Dataset	13
5.2	Ambiente hardware e software	13
5.3	Misura dei tempi e della memoria	13

6	Risultati e confronto	13
6.1	Tempo di esecuzione	13
6.2	Consumo di memoria	13
6.3	Scalabilità	13
6.4	Needleman-Wunsch vs Hirschberg	13

1 Introduzione

2 Definizione del Global Alignment problem e dei prerequisiti essenziali

2.1 Rappresentazione delle sequenze biologiche

Il DNA è costituito da quattro tipi di nucleotidi, caratterizzati dalle seguenti basi azotate:

- Adenina, A ;
- Citosina, C ;
- Guanina, G ;
- Timina, T .

I due filamenti della molecola sono complementari e antiparalleli, con appaiamenti

$$A \longleftrightarrow T \quad , \quad C \longleftrightarrow G$$

Dal punto di vista computazionale, un singolo filamento può essere rappresentato come una stringa sull'alfabeto

$$\Sigma_{\text{DNA}} = \{A, C, G, T\}.$$

Quindi una stringa di lunghezza n ha forma

$$x = x_1 x_2 \cdots x_n, \quad x_i \in \Sigma_{\text{DNA}} \quad \forall i \in \{1, \dots, n\}.$$

Una *substring* è una porzione contigua di una stringa. Data una stringa di lunghezza n esiste una substring di lunghezza k per $k \leq n$.

Un *k-mero* è una substring di lunghezza $k \geq 1$.

Una *subsequence* è invece una nuova stringa ottenuta eliminando 0 o più elementi della stringa originale mantenendone l'ordine relativo.

Mostriamo un esempio: sia s una stringa che rappresenta una sequenza genomica,

$$s := \text{ACCT},$$

allora **AC** può essere sia una substring che una subsequence, mentre **AT** è definibile soltanto come subsequence.

Due sequenze genomiche omologhe possono differire tra loro a causa di mutazioni. Consideriamo tre eventi possibili associati a tali mutazioni:

$$\begin{array}{ccc} \text{AC} & \xrightarrow{\text{sostituzione}} & \text{AA} \\ \text{AC} & \xrightarrow{\text{inserzione}} & \text{AAC} \\ \text{AAC} & \xrightarrow{\text{delezione}} & \text{AC} \end{array}$$

Gli ultimi due eventi verranno ogni tanto trattati come appartenenti a un unico tipo, dal nome *indel*.

Supponiamo, quindi, di avere due sequenze genomiche omologhe che differiscono a causa di mutazioni genetiche; come possiamo quantificare e individuare la somiglianza tra le due sequenze?

2.2 Allineamento e modello di scoring

Stiamo quindi iniziando a definire il problema di nostro interesse.

Dato che un nostro obiettivo è quello di misurare la differenza tra due sequenze genetiche, trattiamo adesso la *distanza di Hamming*, che è lo strumento più semplice per valutare la somiglianza tra due stringhe. Siano x, y due stringhe della stessa lunghezza, allora la distanza di Hamming è

$$d_H(x, y) := |\{i : x_i \neq y_i\}|,$$

ovvero il numero di posizioni in cui due stringhe differiscono. La distanza di Hamming è però applicabile soltanto a stringhe della stessa lunghezza e non modella adeguatamente inserzioni e delezioni, che possono inoltre alterare la corrispondenza tra le posizioni delle due sequenze.

Abbiamo quindi bisogno di una misura più generale. Definiamo allora l'*edit distance*, o anche distanza di Levenshtein, $d(v, w)$, come il numero minimo di operazioni elementari di inserzione, delezione e sostituzione necessarie per trasformare una sequenza v in una sequenza w .

L'edit distance fornisce una misura della dissimilarità tra due sequenze. Per rappresentare esplicitamente le corrispondenze tra i loro simboli introduciamo ora il concetto di **allineamento**.

Siano v e w due sequenze. Un allineamento di v e w è una matrice

$$\mathcal{A} \in (\Sigma_{\text{DNA}} \cup \{-\})^{2 \times L},$$

con,

$$\max\{|v|, |w|\} \leq L \leq |v| + |w|,$$

tale che, eliminando i simboli di gap dalla prima riga, si ottiene v , mentre eliminandoli dalla seconda si ottiene w . Non sono ammesse colonne contenenti due gap.

Ogni colonna di un allineamento corrisponde a uno dei seguenti casi:

- match;
- mismatch (sostituzione);
- inserzione;
- delezione;

Visualizziamo cosa è stato appena detto: siano v_{aligned} e w_{aligned} le sequenze allineate e $\mathcal{A}$ il loro allineamento, allora,

$$\begin{array}{rcl} v_{\text{aligned}} & : & A \quad C \quad T \quad - \quad G \quad C \quad A \\ w_{\text{aligned}} & : & A \quad - \quad T \quad C \quad G \quad C \quad A \end{array} = \mathcal{A}$$

Per valutare la similarità tra simboli introduciamo una funzione di **scoring**,

$$\delta : ((\Sigma_{\text{DNA}} \cup \{-\})^2 \setminus \{(-, -)\}) \rightarrow \mathbb{R},$$

che, dati due simboli a e b , restituisce il contributo di score associato al loro allineamento.

Una funzione di scoring (anche detta modello) per una sequenza di DNA è generalmente definita attribuendo un premio in caso di match e una penalità in caso contrario

$$\delta(a, b) = \begin{cases} +\mu, & a = b, & a, b \in \Sigma_{\text{DNA}} \\ -\eta, & a \neq b, & a, b \in \Sigma_{\text{DNA}} \\ -\sigma, & \text{se esattamente uno tra } a, b \text{ è un gap} \end{cases}$$

con $\mu, \eta, \sigma > 0$. Il modello in cui mismatch e gap hanno la stessa penalità si ottiene come caso particolare ponendo $\eta = \sigma$.

Grazie alla funzione di scoring possiamo anche definire la **scoring matrix**,

δ	A	C	G	T	$-$
A	$+\mu$	$-\eta$	$-\eta$	$-\eta$	$-\sigma$
C	$-\eta$	$+\mu$	$-\eta$	$-\eta$	$-\sigma$
G	$-\eta$	$-\eta$	$+\mu$	$-\eta$	$-\sigma$
T	$-\eta$	$-\eta$	$-\eta$	$+\mu$	$-\sigma$
$-$	$-\sigma$	$-\sigma$	$-\sigma$	$-\sigma$	n.a.

Siano x, y le sequenze allineate corrispondenti rispettivamente a v e w , quindi $x_i, y_i \in \Sigma_{\text{DNA}} \cup \{-\}$, $\forall i \in \{1, \dots, L\}$.

Sia

$$\mathcal{A} = \begin{pmatrix} x_1 & \cdots & x_L \\ y_1 & \cdots & y_L \end{pmatrix},$$

allora, definiamo lo score complessivo dell'allineamento come

$$\text{score}(\mathcal{A}) = \sum_{i=1}^L \delta(x_i, y_i).$$

Nel modello precedente ogni gap viene penalizzato indipendentemente. Una successione di gap consecutivi può tuttavia essere interpretata come risultato di un unico evento di inserzione o delezione. Per questo motivo vengono spesso utilizzate penalità affini, che distinguono il costo di apertura di un gap da quello della sua estensione. Tale generalizzazione richiede stati aggiuntivi nella programmazione dinamica; nel seguito ci concentreremo tuttavia sul modello a penalità lineare utilizzato dagli algoritmi oggetto del confronto.

2.3 Programmazione dinamica

La *programmazione dinamica* è una tecnica algoritmica adatta a problemi caratterizzati da sottoproblemi sovrapposti e da una proprietà di sottostruttura ottima. I risultati dei sottoproblemi vengono memorizzati, evitando di calcolarli più volte.

Nel problema di allineamento che considereremo, i sottoproblemi possono essere indicizzati mediante due indici i e j . In particolare, una ricorrenza avrà tipicamente forma

$$P(i, j) = F(P(i-1, j), P(i, j-1), P(i-1, j-1)).$$

Siano $|v| = n$ e $|w| = m$. Indicheremo con

$$S_{i,j}$$

il valore ottimo relativo ai prefissi

$$v_1 \cdots v_i \quad ew_1 \cdots w_j,$$

con $0 \leq i \leq n$ e $0 \leq j \leq m$

Nel caso dell'allineamento di sequenze, lo stato $S_{i,j}$ dipenderà dai tre stati

$$S_{i-1,j}, \quad S_{i,j-1}, \quad S_{i-1,j-1},$$

che corrisponderanno rispettivamente a una delezione, un'inserzione e a un match o mismatch.

Memorizzando esplicitamente tutti gli stati è necessaria una matrice di $(n+1)(m+1)$ elementi, e quindi uno spazio $O(nm)$

Una volta calcolato il valore ottimo, una soluzione corrispondente può essere ricostruita memorizzando le scelte effettuate e percorrendole a ritroso mediante *traceback*

2.4 Longest Common Subsequence

Una *common subsequence* di due stringhe v e w è una stringa che è una subsequence sia di v sia di w .

Quindi, siano

$$v = v_1 v_2 \cdots v_n, \quad w = w_1 w_2 \cdots w_m.$$

Una stringa

$$z = z_1 z_2 \cdots z_k$$

è una common subsequence di v e w se esistono indici

$$1 \leq i_1 < i_2 < \cdots < i_k \leq n$$

e

$$1 \leq j_1 < j_2 < \cdots < j_k \leq m$$

tali che

$$z_t = v_{i_t} = w_{j_t}, \quad \forall t \in \{1, \dots, k\}$$

Facciamo un esempio per chiarire:

Siano

$$v := \text{ACCT}, \quad w := \text{ACT}.$$

Allora, la stringa

$$z = \text{ACT}$$

è una common subsequence.

Tra tutte le common subsequence di v e w , siamo interessati a quella di lunghezza massima, detta **Longest Common Subsequence** (LCS).

Indichiamo con $s(v, w)$ la lunghezza di una LCS tra v e w :

$$s(v, w) = \max\{|z| : z \text{ è una common subsequence di } v \text{ e } w\}$$

Considerando una variante dell'edit distance in cui sono ammesse soltanto inserzioni e delezioni, trasformare v in w utilizzando una common subsequence di lunghezza k richiede

$$(n - k) + (m - k)$$

operazioni. Per una LCS, con $k = s(v, w)$, si ottiene quindi

$$d_{\text{indel}}(v, w) = n + m - 2s(v, w)$$

Il problema della LCS può quindi essere ricondotto alla ricerca di un cammino di peso massimo da $(0, 0)$ a (n, m) .

A ogni coppia di prefissi

$$v_1 \cdots v_i, \quad w_1 \cdots w_j$$

associamo un nodo (i, j) .

Da ogni nodo sono possibili tre tipi di movimento:

- $(i - 1, j) \rightarrow (i, j)$;
- $(i, j - 1) \rightarrow (i, j)$;
- $(i - 1, j - 1) \rightarrow (i, j)$, soltanto quando $v_i = w_j$.

Nel problema della LCS assegniamo un peso a ogni arco, $w(e)$, dove $e \in E$, con E insieme degli archi della griglia.

$$w(e) = \begin{cases} 1, & \text{arco diagonale corrispondente a un match,} \\ 0, & \text{arco verticale o orizzontale} \end{cases}$$

Il problema della LCS può quindi essere ricondotto alla ricerca di un cammino di peso massimo da $(0, 0)$ a (n, m) nel DAG associato alla griglia descritta sopra.

Sia $S_{i,j}$ la lunghezza di una LCS tra $v_1 \cdots v_i$ e $w_1 \cdots w_j$. Siano

$$S_{i,0} = 0, \quad S_{0,j} = 0.$$

Allora possiamo così definire la ricorrenza:

$$S_{i,j} = \max \begin{cases} S_{i-1,j} \\ S_{i,j-1} \\ S_{i-1,j-1} + 1 \quad \text{se } v_i = w_j \end{cases}$$

I primi due termini corrispondono rispettivamente a non utilizzare v_i oppure w_j nella subsequence comune. Se invece $v_i = w_j$, è possibile utilizzare entrambi i simboli aggiungendo 1 alla soluzione relativa ai prefissi precedenti. La struttura della ricorrenza riprende quindi la dipendenza dai tre sottoproblemi introdotti nella sezione precedente.

Poiché gli stati sono $(n+1)(m+1)$, memorizzare l'intera matrice richiede spazio

$$O(nm).$$

Poiché ogni stato viene calcolato utilizzando un numero costante di operazioni, anche il tempo di calcolo è

$$O(nm).$$

Se interessa soltanto la lunghezza della LCS, è sufficiente mantenere in memoria le righe necessarie al calcolo, riducendo così lo spazio a $O(\min(n, m))$.

2.5 Alignment graph

La rappresentazione su grafo introdotta per la LCS può essere generalizzata al problema dell'allineamento di sequenze.

Siano v e w due sequenze, con $|v| = n$ e $|w| = m$. Definiamo l'**alignment graph** come il grafo diretto aciclico i cui nodi sono le coppie

$$(i, j), \quad 0 \leq i \leq n, \quad 0 \leq j \leq m,$$

dove il nodo (i, j) rappresenta l'allineamento dei prefissi

$$v_1 \cdots v_i, \quad w_1 \cdots w_j.$$

Lo stato iniziale è $(0, 0)$ e quello finale è (n, m) .

Ogni allineamento tra v e w corrisponde a un cammino nell'alignment graph da $(0, 0)$ a (n, m) e, viceversa, ogni cammino tra questi nodi rappresenta un possibile allineamento.

Le mosse hanno interpretazione:

Mossa	Arco	Operazione nell'allineamento
verticale	$(i-1, j) \rightarrow (i, j)$	delezione / v_i contro gap
orizzontale	$(i, j-1) \rightarrow (i, j)$	inserzione / gap contro w_j
diagonale	$(i-1, j-1) \rightarrow (i, j)$	match o mismatch

A ogni mossa è associata, quindi, un arco del grafo. Il peso di ogni arco è definito come segue:

$$\begin{aligned}w((i-1, j) \rightarrow (i, j)) &= \delta(v_i, -), \\w((i, j-1) \rightarrow (i, j)) &= \delta(-, w_j), \\w((i-1, j-1) \rightarrow (i, j)) &= \delta(v_i, w_j),\end{aligned}$$

con δ funzione di scoring.

Il peso di un cammino coincide quindi con lo score dell'allineamento corrispondente.

2.6 Global Alignment Problem

Definiamo ora formalmente il problema dell'allineamento globale, che costituisce il problema di riferimento per gli algoritmi analizzati nel seguito.

Siano v e w due sequenze e sia δ la funzione di scoring definita precedentemente.

Dato un allineamento

$$A = \begin{pmatrix} x_1 & x_2 & \cdots & x_r \\ y_1 & y_2 & \cdots & y_r \end{pmatrix}$$

costituito da r colonne, definiamo il suo score complessivo come

$$\text{score}(A) = \sum_{k=1}^r \delta(x_k, y_k)$$

Sia $\mathcal{A}(v, w)$ l'insieme di tutti i possibili allineamenti tra v e w . Il **Global Alignment Problem** consiste nel determinare un allineamento di score massimo, ovvero

$$\text{OPT}(v, w) = \max_{A \in \mathcal{A}(v, w)} \text{score}(A)$$

Nel modello di scoring lineare considerato di seguito,

$$\text{match} = +1, \quad \text{mismatch} = -\eta, \quad \text{indel} = -\sigma,$$

con $\eta, \sigma > 0$

Lo score di un allineamento può quindi essere espresso come

$$\text{score}(A) = \# \text{match} - \eta \# \text{mismatch} - \sigma \# \text{gap}.$$

dove $\# \text{gap}$ indica il numero di colonne dell'allineamento contenenti un gap dovuto a una trasformazione indel.

Per quanto osservato nella sezione precedente, ogni allineamento corrisponde a un cammino da $(0, 0)$ a (n, m) nell'alignment graph e il peso del cammino coincide con lo score dell'allineamento. Il Global Alignment Problem può quindi essere interpretato come la ricerca di un cammino di peso massimo tra questi due nodi.

Da questa formulazione emergono tre obiettivi computazionali che guideranno l'analisi degli algoritmi successivi:

- calcolare il valore ottimo $\text{OPT}(v, w)$;
- ricostruire un allineamento che realizzi tale valore;
- ricostruire un allineamento ottimo riducendo l'utilizzo di memoria.

3 Algoritmo di Needleman-Wunsch

3.1 Ricorrenza, inizializzazione e traceback

Sia $S_{i,j}$ il miglior score globale tra $v_1 \cdots v_i$ e $w_1 \cdots w_j$, con $0 \leq i \leq n, 0 \leq j \leq m$. Quindi, come visto nel precedente capitolo, possiamo affermare che $S_{i,j}$ coincide con il peso di un cammino massimo da $(0,0)$ a (i,j) .

Consideriamo l'ultima colonna di un allineamento ottimo, abbiamo tre possibilità:

- **Delezione**

L'ultima colonna è

$$\begin{pmatrix} v_i \\ - \end{pmatrix}$$

quindi prima di essa è stato allineato $v_1 \cdots v_{i-1}$ con $w_1 \cdots w_j$.

Il contributo è

$$S_{i-1,j} + \delta(v_i, -)$$

- **Inserzione**

L'ultima colonna è

$$\begin{pmatrix} - \\ w_j \end{pmatrix}$$

quindi prima di essa è stato allineato $v_1 \cdots v_i$ con $w_1 \cdots w_{j-1}$.

Il contributo è

$$S_{i,j-1} + \delta(-, w_j)$$

- **Match o mismatch**

L'ultima colonna è

$$\begin{pmatrix} v_i \\ w_j \end{pmatrix}$$

quindi prima di essa è stato allineato $v_1 \cdots v_{i-1}$ con $w_1 \cdots w_{j-1}$.

Il contributo è

$$S_{i-1,j-1} + \delta(v_i, w_j)$$

Per gli stati interni dell'allineamento, con $1 \leq i \leq n$ e $1 \leq j \leq m$, possiamo formulare la ricorrenza a partire dai tre possibili predecessori. Prima però è necessario inizializzare gli stati sul bordo della matrice, che corrispondono con i casi base del problema dell'allineamento globale:

$$S_{0,0} = 0;$$

$$S_{i,0} = S_{i-1,0} + \delta(v_i, -) = -i\sigma;$$

$$S_{0,j} = S_{0,j-1} + \delta(-, w_j) = -j\sigma.$$

A differenza della LCS, gli stati sul bordo non hanno valore nullo: nel global alignment l'intero prefisso deve essere allineato. Allineare $v_1 \cdots v_i$ con la stringa vuota richiede quindi i gap, mentre allineare la stringa vuota con $w_1 \cdots w_j$ ne richiede j .

Possiamo, quindi, formalizzare la *ricorrenza*:

$$S_{i,j} = \max \begin{cases} S_{i-1,j} + \delta(v_i, -), \\ S_{i,j-1} + \delta(-, w_j), \\ S_{i-1,j-1} + \delta(v_i, w_j) \end{cases}$$

con $1 \leq i \leq n$ e $1 \leq j \leq m$.

Questa relazione costituisce la ricorrenza fondamentale dell'algoritmo di Needleman-Wunsch.

Il calcolo di $S_{n,m}$ soddisfa il primo degli obiettivi formulati nel capitolo precedente, fornendo

$$S_{n,m} = \text{OPT}(v, w).$$

Per ricostruire anche un allineamento che realizzi tale score è necessario mantenere l'informazione relativa alle scelte effettuate durante il calcolo. Quindi, durante il calcolo di $S_{i,j}$, possiamo memorizzare quale predecessore ha prodotto il massimo all'interno di una matrice B :

$$B_{i,j} \in \{\uparrow, \leftarrow, \nearrow\}.$$

Sul bordo della matrice il predecessore è univocamente determinato:

$$\begin{aligned} B_{i,0} &= \uparrow, & i &= 1, \dots, n, \\ B_{0,j} &= \leftarrow, & j &= 1, \dots, m. \end{aligned}$$

A ciascuna direzione memorizzata in $B_{i,j}$ corrisponde una colonna dell'allineamento secondo la seguente associazione:

$B_{i,j}$	colonna dell'allineamento
$\uparrow$	$(v_i, -)$
$\leftarrow$	$(-, w_j)$
$\nearrow$	(v_i, w_j)

Così facendo, partendo da (n, m) possiamo ricostruire l'allineamento percorrendo i predecessori fino a $(0, 0)$. Notiamo però che tramite il *backtracking* le colonne vengono generate in ordine contrario, quindi al termine vanno invertite.

In caso di parità tra più transizioni esistono più cammini, e quindi potenzialmente più allineamenti ottimi. Se l'obiettivo è ricostruire un singolo allineamento ottimo, è sufficiente adottare una regola deterministica di tie-breaking; per rappresentare tutti gli allineamenti ottimi sarebbe invece necessario conservare tutti i predecessori che realizzano il massimo.

3.2 Pseudocodice

Possiamo adesso scrivere lo pseudocodice per l'algoritmo di Needleman-Wunsch.

Listing 1: Global Alignment

```
NeedlemanWunsch(v,w,delta)
  n <- |v|
  m <- |w|

  // inizializzazione
  S[0,0] <- 0
  for i <- 1,...,n
    S[i,0] <- S[i-1,0] + delta(v[i], '-')
    B[i,0] <- up
  for j <- 1,...,m
    S[0,j] <- S[0,j-1] + delta('-', w[j])
    B[0,j] <- left

  // ricorrenza
  for i <- 1,...,n
    for j <- 1,...,m
      up <- S[i-1,j] + delta(v[i], '-')
      left <- S[i,j-1] + delta('-', w[j])
      diagonal <- S[i-1,j-1] + delta(v[i], w[j])
```

```

        S[i,j] <- max(up, left, diagonal)
        B[i,j] <- predecessore che ha prodotto il massimo

// ricostruzione dell'allineamento
alignment <- Backtrack(B,v,w,n,m)

return (S[n,m] , alignment)

```

Adottiamo una politica deterministica di tie-breaking che assegna priorità, nell'ordine, allo spostamento diagonale, verticale e orizzontale. Possiamo quindi scrivere lo pseudocodice del traceback in questa maniera.

Listing 2: Ricostruzione dell'allineamento tramite traceback

```

Backtrack(B,v,w,i,j)
  alignedV <- empty
  alignedW <- empty

  while i > 0 or j > 0
    if B[i,j] = diagonal
      append v[i] to alignedV
      append w[j] to alignedW
      i <- i - 1
      j <- j - 1

    else if B[i,j] = up
      append v[i] to alignedV
      append '-' to alignedW
      i <- i - 1

    else
      append '-' to alignedV
      append w[j] to alignedW
      j <- j - 1

  reverse alignedV
  reverse alignedW

  return (alignedV,alignedW)

```

Quindi, la prima procedura costruisce la matrice degli score e quella dei predecessori; la seconda utilizza quest'ultima per ricostruire un allineamento ottimo a partire dalla cella (n, m) .

3.3 Analisi della complessità

Abbiamo allora definito l'algoritmo standard di Needleman-Wunsch. Possiamo adesso analizzarne la complessità, centro del nostro confronto.

Sia $T(n, m)$ la complessità di esecuzione, temporale, dell'algoritmo. Sia $S(n, m)$ la complessità spaziale dell'algoritmo.

Allora,

$$T(n, m) = \Theta(nm),$$

poiché la matrice contiene $(n + 1)(m + 1)$ stati e ciascuno viene calcolato mediante un numero costante di operazioni.

Memorizzando l'intera matrice avremo

$$S(n, m) = \Theta(nm).$$

Pertanto, per sequenze di lunghezza dello stesso ordine, tempo e spazio crescono quadraticamente rispetto alla lunghezza.

La crescita quadratica dello spazio può quindi diventare un limite per sequenze di grandi dimensioni, poiché memorizzare l'intera matrice richiede una quantità di memoria proporzionale al prodotto delle lunghezze delle due sequenze.

Inoltre, non abbiamo ancora considerato il costo del traceback. Un cammino da (n, m) a $(0, 0)$ contiene al più $n + m$ archi, quindi, il traceback percorre al massimo $n + m$ celle, richiedendo tempo

$$T_{\text{traceback}}(n, m) = O(n + m),$$

Inoltre, l'allineamento prodotto può contenere al massimo $n + m$ colonne, e richiede quindi

$$O(n + m)$$

spazio per la memorizzazione dell'output.

Pertanto, il costo temporale complessivo dell'algoritmo rimane

$$T(n, m) = \Theta(nm) + O(n + m) = \Theta(nm),$$

mentre la complessità spaziale complessiva della versione standard è

$$S(n, m) = \Theta(nm) + O(n + m) = \Theta(nm).$$

Sappiamo che, per calcolare

$$S_{i,j}$$

abbiamo bisogno di

$$S_{i-1,j}, \quad S_{i,j-1}, \quad S_{i-1,j-1}.$$

Allora, non è difficile notare come il calcolo della colonna j richiede soltanto la colonna $j - 1$ e i valori già calcolati della colonna j . Quindi, se riduciamo il problema soltanto al calcolo del punteggio ottimo, potremmo diminuire lo spazio necessario, andando ad usare soltanto due vettori di $n + 1$ o $m + 1$ elementi, scegliendo opportunamente la sequenza più corta, ottenendo quindi uno spazio lineare.

$$S_{\text{score}}(n, m) = \Theta(\min\{n, m\}).$$

Notiamo che la complessità nel tempo non cambia.

Tale ottimizzazione consente quindi di soddisfare il primo obiettivo computazionale, ossia il calcolo dello score ottimo, utilizzando spazio lineare. Tuttavia, eliminando le celle già utilizzate vengono perse anche le informazioni necessarie per effettuare il traceback e ricostruire un allineamento ottimo. Il problema diventa quindi quello di ottenere anche la soluzione, e non soltanto il suo valore, mantenendo una complessità spaziale lineare.

3.4 Implementazione

4 Algoritmo di Hirschberg

4.1 Ricorrenza, inizializzazione e traceback

4.2 Pseudocodice

4.3 Analisi della complessità

4.4 Implementazione

5 Metodologia sperimentale

5.1 Dataset

5.2 Ambiente hardware e software

5.3 Misura dei tempi e della memoria

6 Risultati e confronto

6.1 Tempo di esecuzione

6.2 Consumo di memoria

6.3 Scalabilità

6.4 Needleman-Wunsch vs Hirschberg